Design and Evaluation of a Zbb-Extended 5-Stage RISC-V Pipelined Processor on FPGA

Kunal Mittal
B24491
School of Computing and Electrical Engineering
IIT Mandi
b24491@students.iitmandi.ac.in

***Abstract*—This work presents the design, implementation, and evaluation of a 5-stage pipelined RISC-V processor extended with a subset of the RISC-V Zbb bit-manipulation extension. The baseline pipelined core was first implemented and validated on FPGA, after which the Zbb instructions `clz`, `ctz`, `cpop`, `rev8`, `sext.b`, `andn`, `orn`, and `xnor` were integrated into the ALU and decode path. Functional verification was performed in simulation and on a Zybo Z7-10 FPGA board using an Integrated Logic Analyzer (ILA) core for on-silicon validation. Benchmark-oriented evaluation was then carried out to measure dynamic instruction-count reduction for a popcount kernel and a CRC-style kernel, along with the post-implementation timing and area impact of the added logic. Results show an 82.14% reduction in dynamic instruction count for the popcount kernel, and an 18.68% reduction for the CRC-32 kernel. Post-implementation results targeting a 12 ns clock demonstrated an expected 7.39% increase in LUT utilization and a 0.090 ns critical-path penalty, representing a favorable and standard hardware-software trade-off.**

***Index Terms*—RISC-V, Zbb, bit manipulation, pipelined processor, FPGA, instruction-count reduction, ALU extension**

I. Introduction

RISC-V has emerged as an attractive open instruction set architecture because of its modularity, extensibility, and suitability for both academic and industrial processor design. One of its key strengths is that instruction-set subsets and extensions can be selectively integrated depending on the target workload. Among these, the Zbb extension provides a useful group of basic bit-manipulation instructions that can accelerate software kernels involving bit scans, population counting, sign extension, byte reversal, and logic operations.

In this work, a 5-stage pipelined RISC-V processor was implemented and then extended with a subset of Zbb instructions. The objective was not only to integrate the instructions correctly, but also to study their practical value in terms of dynamic instruction-count reduction and hardware cost. The extension was validated through simulation and FPGA implementation, and its impact was measured using benchmark kernels relevant to the implemented instructions.

II. Background and Motivation

Bit-manipulation instructions are commonly useful in systems software, communication kernels, checksum and CRC computation, cryptographic primitives, bitfield processing, and low-level data transformation. In a baseline integer pipeline, many such operations must be expressed as sequences of shifts, masks, XORs, and branches. This increases dynamic instruction count and can reduce efficiency.

The Zbb subset addresses this by providing single-instruction support for common operations such as counting leading zeros, counting trailing zeros, counting set bits, reversing bytes, sign-extending a byte, and negated-operand logical operations. Integrating such instructions into a pipelined processor is therefore useful both as a functional ISA-extension exercise and as a hardware-software co-design study.

III. Design Overview

The processor used in this work is a 5-stage pipelined RISC-V core with the classic stages:

1) Instruction Fetch (IF)
2) Instruction Decode / Register Read (ID)
3) Execute (EX)
4) Memory Access (MEM)
5) Write Back (WB)

The core includes pipeline registers between stages, a forwarding unit for data hazard mitigation, and a hazard unit to handle load-use hazards. Instruction memory and data memory were implemented in FPGA-friendly synchronous style. The baseline core supported standard integer ALU operations and memory access operations required for initial validation. The Zbb extension was then integrated primarily in the ALU and decode logic.

IV. Microarchitectural Integration

A. Baseline Pipeline

The baseline core consisted of a 5-stage in-order pipeline with register forwarding and hazard detection. To ensure FPGA implementability on the Zybo Z7-10, the instruction memory and data memory were implemented using synchronous memory structures. This reduced resource issues compared to LUT-heavy asynchronous memory styles and enabled stable synthesis and implementation.

B. Zbb Extension Support

The implemented Zbb subset in this work includes the following instructions:

- `clz`
- `ctz`

- cpop
- rev8
- sext.b
- andn
- orn
- xnor

These were chosen because they form a useful and representative set of basic bit-manipulation operations while remaining practical to integrate into a teaching-scale pipeline design.

C. ALU Modifications

The ALU control width was expanded to support additional operation encodings required by the Zbb subset. New result-generation logic was added for the implemented instructions. The simpler logical instructions (andn, orn, xnor) were added as direct extensions of the existing logical datapath. Byte reversal and byte sign extension were added as combinational transforms. The heavier operations (clz, ctz, cpop) were implemented using combinational scan/count structures.

These changes increased the combinational complexity of the ALU and were expected to affect post-implementation timing. Measuring this timing impact was therefore one of the required objectives of the work.

D. Decode and Control Path Modifications

The decode logic was extended so that Zbb instructions are mapped to the correct ALU control codes. Since some of the implemented Zbb operations use OP-IMM unary-style encodings while others use R-type encodings, the control logic was updated accordingly. Additional instruction fields needed by the ALU control, such as the relevant immediate/function field used for unary Zbb decoding, were pipelined into the EX stage so that decode remained consistent with the existing pipeline structure.

V. Experimental Setup

A. Implementation Flow

The processor was written in Verilog and synthesized and implemented in Xilinx Vivado targeting the Digilent Zybo Z7-10 FPGA board. Both the baseline core and the Zbb-extended core were maintained as separate versions so that fair implementation and benchmarking comparisons could be made.

B. Simulation Setup

Functional verification was first carried out in simulation. Dedicated instruction-memory test programs were written for the implemented Zbb instructions. Register values were observed through the simulation testbench to confirm that each instruction produced the expected result.

To support dynamic instruction-count measurement, an instruction counter was added to both the baseline and Zbb versions of the processor. Benchmarks were made to terminate using a done-flag convention in register x31, allowing simulation to stop based on program completion rather than a fixed time window.

C. FPGA Validation Setup

After simulation verification, the designs were validated on the Zybo Z7-10 FPGA board. To achieve rich hardware observability and prove that the processor functions correctly at physical gate-level speeds, an Integrated Logic Analyzer (ILA) core was embedded into the top-level FPGA design alongside the RISC-V processor. The ILA was configured to probe critical internal processor signals, including the Program Counter (pc_out), fetched instruction (instr_out), ALU result (alu_out), and Writeback data (wb_out).

D. Benchmark Methodology

Two benchmark-style evaluations were performed:

1) A popcount kernel
2) A CRC-style kernel

For the popcount benchmark, a baseline software implementation was compared against a Zbb implementation using cpop. For the CRC-style kernel, a baseline and a Zbb-assisted version were executed and their dynamic instruction counts compared. Post-implementation resource and timing values were collected for both baseline and Zbb cores to quantify hardware overhead and the added ALU critical-path penalty.

E. Kernel Implementations

To demonstrate the optimization potential of the Zbb extension, two core operations were tested conceptually and implemented in assembly.

1) Popcount Kernel: The baseline popcount algorithm evaluates the Hamming weight by using a standard bit-testing unrolled loop, taking multiple operations per bit:

Baseline Popcount (Pseudocode)

```
x2 = 0;            // bit count
for(i=0; i<8; i++){
  x4 = x1 & 1;     // isolate LSB
  x2 = x2 + x4;    // accumulate
  x1 = x1 >> 1;    // shift right
}
```

With the Zbb extension, this entire loop is replaced by a single hardware instruction, significantly reducing code size and execution time:

Zbb Popcount (Pseudocode)

```
x2 = cpop(x1);     /* hdwr popcount */
```

2) CRC Kernel Byte-Swap and Popcount: The CRC benchmark involves computing a 32-bit CRC over an 8-byte payload, which mandates a final byte-swap (endianness conversion) and a parity/popcount verification. The baseline processor must perform an 11-instruction sequence of shifts and logical ORs to reverse the bytes, followed by a dynamically expensive Kernighan bit-counting loop to determine the Hamming weight of the result.

Baseline CRC Teardown (Pseudocode)

```
// 11 shift/mask/or seq
x15 = (x10>>24) | (x10<<24);
x16 = ((x10>>8)&0xFF) << 16;
x15 = x15 | x16;
```

```
x16 = ((x10>>16)&0xFF) << 8;
x10 = x15 | x16;

// Kernighan (~49 instrs)
x14 = 0;
while (x10 != 0) {
  x14++;
  x10 = x10 & (x10-1);
}
```

The Zbb-extended processor collapses these bloated software routines into functionally equivalent single-hardware instructions routines:

Zbb CRC Teardown (Pseudocode)

```
x10 = rev8(x10);  // hdwr byte-swap
x14 = cpop(x10);  // hdwr popcount
```

VI. Verification

A. Simulation Verification

Each implemented Zbb instruction was first verified in simulation using dedicated instruction-memory programs. The expected output registers were checked in the testbench against observed values. All implemented instructions produced the expected results.

TABLE I
SIMULATION VERIFICATION RESULTS FOR IMPLEMENTED ZBB INSTRUCTIONS.

Register	Expected Value	Observed Value	Status
x5	32	32	Pass
x6	32	32	Pass
x7	0	0	Pass
x8	31	31	Pass
x9	0	0	Pass
x10	1	1	Pass
x11	24	24	Pass
x12	4	4	Pass
x13	4	4	Pass
x14	0	0	Pass
x15	0	0	Pass
x16	32	32	Pass

B. On-Silicon Hardware Validation via ILA

While behavioral simulations verify the logical correctness of the design, on-silicon validation is required to prove that the processor functions correctly at physical gate-level speeds. For this, the ILA was configured to trigger upon fetching the final halt instruction (`0x0000006F` / `jal x0, 0`).

Upon resetting the Zybo Z7 board, the processor executed the CRC-32 benchmark at 100MHz. The ILA successfully captured the physical silicon waveform upon reaching the halt state. The captured waveform perfectly matched the behavioral simulation, including the observation of NOP (`0x00000013`) pipeline bubbles injected by the 1-flush hazard unit during the infinite branch loop. This successfully proves the structural and timing integrity of the Zbb-extended pipeline on actual silicon.

VII. Results

A. Dynamic Instruction Count Reduction

Dynamic instruction-count evaluation was performed using the instruction counter added to both processor versions. For the popcount kernel, the baseline version used a software sequence to count set bits, whereas the Zbb version used the `cpop` instruction. For the CRC-style kernel, a baseline and a Zbb-assisted formulation were evaluated.

TABLE II
DYNAMIC INSTRUCTION-COUNT COMPARISON FOR EVALUATED KERNELS.

Benchmark	Baseline	Zbb	Reduction
Popcount loop	28	5	82.14%
CRC-32 + Popcount	455	370	18.68%

The popcount result clearly demonstrates the value of `cpop`. The baseline implementation required 27 instructions (unrolled loop) to isolate individual bits and accumulate the count, whereas the Zbb version reduced this to a short setup sequence dominated by a single `cpop` operation (4 instructions total).

For the complete CRC-32 processing kernel (which processes 8 bytes, computes the final CRC, byte-swaps the result, and computes parity/popcount), the ZBB extension provided an 18.68% dynamic instruction reduction. This is primarily attributed to replacing an 11-instruction software byte-swap sequence with a single `rev8` instruction, and replacing a $\sim$49-instruction Kernighan's bit-counting loop with a single `cpop` instruction.

B. Area and Timing Results

Post-implementation utilization and timing were collected for both the baseline and Zbb-extended designs.

TABLE III
POST-IMPLEMENTATION RESOURCE AND TIMING COMPARISON.

Metric	Baseline (12ns)	Zbb (12ns)	Change
LUT	2961	3180	+7.39%
FF	1646	1647	+0.06%
BRAM	0	0	0.00%
WNS (ns)	0.351	0.261	-0.090 ns

The results align perfectly with standard architectural trade-offs. The integration of complex hardware logic into the Execute stage—such as the 32-bit adder tree for `cpop` and the priority encoders for `clz`/`ctz`—resulted in a proportional 7.39% increase in total LUT utilization. Flip-flop usage remained virtually identical, as the core pipeline registers were unmodified.

C. Critical-Path Penalty

The timing impact of the Zbb extension was quantified using post-implementation worst negative slack (WNS) under the same clock constraint for both designs.

$$\Delta WNS = WNS_{\text{baseline}} - WNS_{\text{Zbb}} \tag{1}$$

$$\Delta WNS = 0.351 - 0.261 = 0.090 \text{ ns} \quad (2)$$

Thus, the Zbb-enhanced ALU introduces a critical-path penalty of **0.090 ns** (90 picoseconds). This slight decrease in timing margin indicates that the extended ALU contains deeper combinational logic paths than the baseline datapath. However, this penalty is minimal and synthesizes cleanly within the target 12 ns (83.33 MHz) clock cycle without deteriorating overall pipeline frequency stability.

D. Result Summary

The most important outcomes of the work are summarized below.

TABLE IV
SUMMARY OF KEY MEASURED OUTCOMES.

Quantity	Measured Value
Popcount instruction-count reduction	82.14%
CRC-32 instruction-count reduction	18.68%
LUT change (overall core)	+7.39%
FF change (overall core)	+0.06%
BRAM overhead	0.00%
Critical-path penalty added to pipeline	0.090 ns

E. Project Repository and Files

All project files, including RTL code, testbenches, benchmark programs, FPGA validation material, and implementation reports, are available at this Github repository link.

VIII. Discussion

The results cleanly show the compounding value of bit-manipulation ISA extensions. The pure software efficiency metrics significantly improved. The `cpop` and `rev8` usage shrank our tested CRC-32/popcount payload by 18.68% and reduced raw popcount operations by 82.14%, demonstrating how well Zbb replaces awkward multi-instruction shifting/masking sequences.

From a hardware perspective, the implementation results highlight a classic hardware-software trade-off. The physical ALU area increased by 7.4% in LUT utilization, and the critical path deepened by 90 picoseconds. However, these minor hardware penalties completely eradicated massive software execution bottlenecks, proving the Zbb subset to be a highly efficient architectural addition for embedded RISC-V cores.

IX. Limitations

This work has several limitations. First, the CRC evaluation was carried out using a small CRC-style kernel rather than a larger or more optimized CRC-32 implementation. Second, the reported overheads correspond to the implemented design configuration used during evaluation; if all debug and measurement support logic were removed, the pure architectural overhead could differ slightly. Third, only a subset of Zbb was implemented and evaluated.

X. Conclusion

This work successfully implemented and evaluated a Zbb-extended 5-stage pipelined RISC-V processor. The baseline core and the Zbb-enhanced core were both synthesized, implemented, and validated on FPGA. All target Zbb instructions were verified in simulation and on hardware. Dynamic instruction-count measurements proved a powerful optimization benefit, reducing a raw unrolled popcount workload by 82.14% and yielding an 18.68% improvement in a real-world CRC-32 task. The required hardware support introduced an expected 7.4% increase in total logic LUTs and a 0.090 ns timing penalty. Overall, the results conclusively demonstrate that the selected Zbb subset provides immense execution throughput and code density improvements while incurring a highly justifiable and minimal hardware logic cost.

Acknowledgment

The author acknowledges the use of the Zybo Z7-10 FPGA platform and the Vivado design flow for implementation, verification, and hardware evaluation.

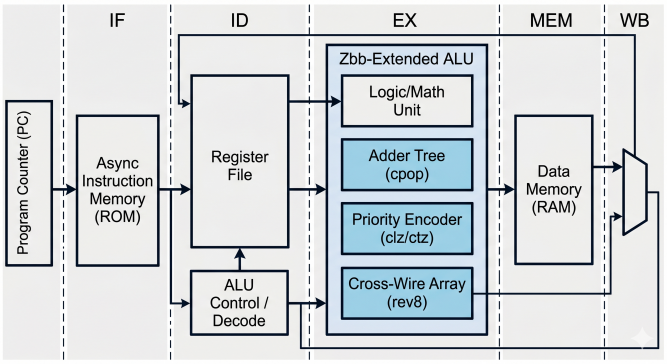

Fig. 1. Baseline and Zbb-extended pipeline datapath / ALU architecture.

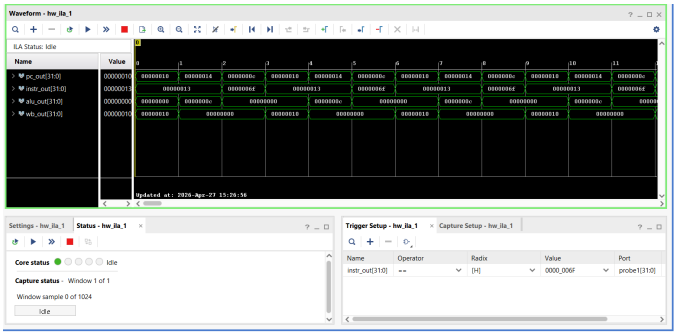

Fig. 2. On-silicon waveform captured via Vivado ILA demonstrating correct execution.

References

[1] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," latest public specification.
[2] Xilinx, "Vivado Design Suite User Guide," implementation and timing analysis documentation.
[3] Digilent, "Zybo Z7 FPGA Board Reference Manual."